

The Islamic University of Gaza
Faculty of Engineering
Department of Computer Engineering



MedLink

Weekly Project Progress Report





MedLink Team

Our team is a cohesive and well-integrated unit, combining diverse skills and perspectives to achieve a common goal. Each member contributes unique strengths, which enables effective collaboration and ensures that all aspects of the project are developed with high quality and consistency.

UI/UX Designer



Sara Atta
220213112

Front-End Developers



Ala'a Eliwa
220211170



Hurih Attallah
220210174

Back-End Developers



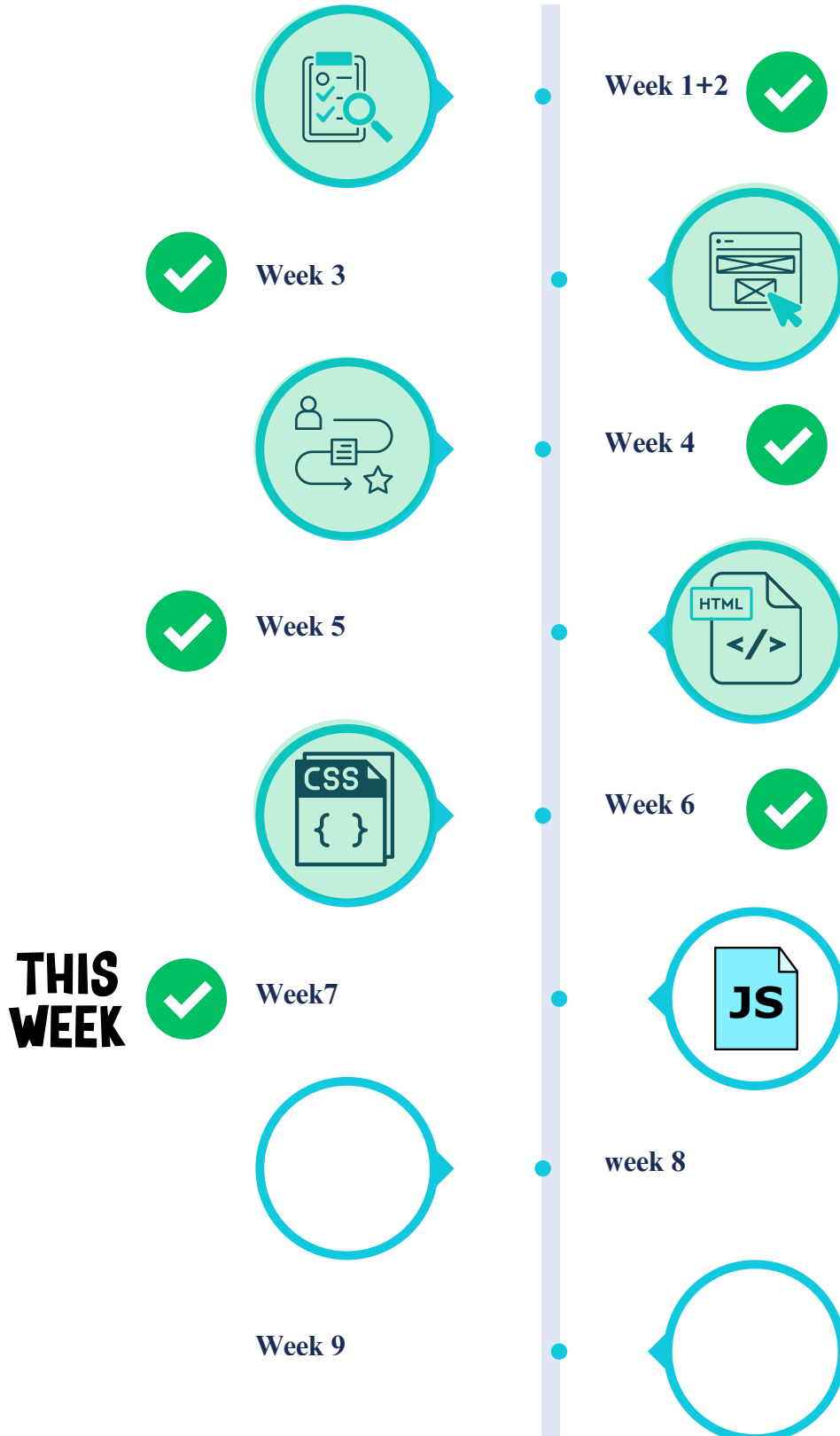
Neeven Zidiah
220213206



Farah Rashid
220210901



Week Number





Work completed this week

To enhance the MedLink platform with dynamic behavior, providing "life" to the static HTML pages and ensuring robust client-side engineering verification through DOM manipulation.



Engineering Verification: Client-side Validation

We prioritized user experience by catching errors before they reach the server, providing immediate visual feedback.

A. Form Integrity & Validation

- Password Matching: In register.js, we implemented logic to compare password fields. If they don't match, the submission is blocked, and the Confirm Password field is highlighted with a red border (#ef4444).
- Required Fields: All critical forms (Login, Register, Request Medicine) prevent empty submissions and provide specific feedback using our custom Toast system.
- Visual Feedback: Fields revert to their default state once the error is corrected, ensuring a "living" form experience.

B. Toast Notification System

We replaced native alert() with a custom MedLinkUI.toast() component in medlink-ui.js:

- Supports success, error, and info types.
- Includes a dynamic progress bar indicating how long the message will stay.
- Uses Glassmorphism and icons for a premium aesthetic.

UI Components & Interactivity

We moved beyond basic browser capabilities to implement custom, high-fidelity UI components.

A. Custom Modals (mlConfirm)

Instead of confirm(), we built a sleek modal system that:

- Blurs the background (backdrop-filter).
- Supports Destructive Actions (e.g., Logout or Delete) by changing icon colors and button styles to red automatically.
- Animates in and out using CSS transitions.

B. Navigation & Sidebars

- Mobile Sidebar/Menu: Implemented a toggle mechanism in citizen-dashboard.js and home.html that handles the visibility of navigation links on small screens.
- Profile Dropdowns: Added logic to open/close profile menus and close them automatically if the user clicks anywhere else on the screen (Outside Click Detection).

C. Interactive Widgets

- Star Rating System: A custom star-rating widget in the pharmacy details page that responds to hover and click events, providing instant visual confirmation of the user's choice.
- Favorite Toggles: "Heart" icons that toggle state (filled vs. outline) instantly, communicating with the localStorage for persistence.

Work completed this week



DOM Manipulation Features

Dynamic Role Switching: The registration form transforms entirely between "Citizen" and "Pharmacy" modes without a page reload.

Search Simulation: A real-time search engine that filters medicine cards as the user types, including "Empty State" handling with a suggestion to "Request Medicine".

Loading States: Every major action (Sign In, Register, Submit Complaint) triggers a fast fa-spinner fa-spin animation on the button to prevent double-clicks and provide feedback.

Features implemented



Structure & Responsiveness (Flexbox/Grid)

Requirement	Status	Key JS File
DOM Manipulation	Completed	register.js, citizen-dashboard.js
Validation	Completed	login.js, register.js
Modals	Completed	medlink-ui.js
Sidebars/Menus	Completed	medlink-ui.js, home.js

Figma Design Link: <https://www.figma.com/design/j8pugP9ert6B7c2p8xbjYV/Untitled?node-id=0-1&t=oiR9N0SdGyyGmbEu-1>

GitHub Repository [Link:https://github.com/alaaeliwa/medlink](https://github.com/alaaeliwa/medlink)

GitHub Pages: <https://alaaeliwa.github.io/medlink/frontend/>



Problems encountered



- Difficulty managing interaction between multiple elements on the same page (DOM complexity).
- Problems with menus closing (Dropdown/Modal) when clicked outside of them.
- Handling of incorrect input cases in forms.
- Double-click button issues.
- Maintaining good performance with increased interactivity.

Solutions applied



- Organize and divide the code into separate JavaScript files for each function.
- Use event listeners thoughtfully (e.g., click outside an element).
- Add clear validation logic before submitting forms.
- Use loading states to prevent duplication.
- Use localStorage to save user states (e.g., Favorites).
- Improve performance by minimizing unnecessary DOM operations.



Plan for next week



-
- Implement AJAX for dynamic data fetching without page reload.
 - Use Fetch/Axios for API communication.
 - Add loading indicators during requests.
 - Implement instant search functionality.
 - Improve overall UX and responsiveness.

Questions for Instructor



-
- Is the level of JavaScript and DOM manipulation appropriate for a university project?
 - Is the code structure (file splitting such as medlink-ui.js, register.js) well-organized?
 - Is the level of UI interactivity sufficient, or do we need to add more elements?
 - Is using localStorage appropriate at this stage, or would it be better to connect it directly to the backend?



MidLink
Web app

